

Trabajo Práctico Unidad 8: Calidad - Análisis de Resultados y Acciones Correctivas

Alumno: Ignacio Figueroa

Materia: Metodología de Sistemas I

Comisión: 7

Institución: Universidad Tecnológica Nacional (UTN) - Tecnicatura Universitaria en Programación

CASO 1 - "Reserva duplicada en horario ocupado"

Paso 1 - Interpretación del resultado

La evidencia nos muestra que el sistema de agenda médica no está validando la disponibilidad de los turnos en tiempo real. Un mismo usuario pudo reservar exactamente el mismo médico (Dra. Molina), en el mismo día y horario dos veces seguidas, y el sistema confirmó ambas operaciones como exitosas generando un duplicado en la base de datos sin lanzar ninguna advertencia.

En este escenario estamos ante una **falla** del sistema. La falla es el comportamiento incorrecto visible (el sistema confirmando un turno duplicado en la interfaz), que es consecuencia directa de un **defecto** en el código (la falta de una restricción o validación lógica que impida insertar un registro con claves idénticas de médico/fecha/hora), originado por un **error** humano del programador al omitir dicha lógica en el backend o en la base de datos.

Paso 2 - Clasificación del defecto

Este defecto se clasifica como un **Error lógico** (y secundariamente de validación en el backend). Justificación: La aplicación procesa los datos, pero la lógica de negocio que dicta que 'un médico solo puede atender a un paciente por vez en un rango horario' no está implementada o está fallando. El log confirma que el POST ingresa limpio dos veces seguidas retornando un estado OK, lo que demuestra que el backend procesa la petición de forma ciega sin verificar el estado previo del recurso.

Paso 3 - Severidad y prioridad

- **Severidad: Alta.** Impacto técnico: Provoca una corrupción de datos en la consistencia de la base de datos (registros duplicados para recursos únicos). Impacto en el uso: Rompe la función principal de una agenda. No es Crítico porque el sistema no se cae ni se bloquea, pero es Alto porque afecta la integridad del negocio.
- **Prioridad: Alta.** Contexto del negocio: El costo de postergación es altísimo. Si esto va a producción, dos pacientes reales se van a presentar al mismo tiempo con el mismo médico, arruinando la atención, generando quejas y rompiendo la confianza en la clínica.

Se debe corregir de inmediato.

Paso 4 - Propuesta de acción correctiva

Debe modificarse el backend y la base de datos. En el backend se debe agregar un control antes de procesar el POST de reserva, que realice una consulta previa para verificar si el médico ya cuenta con un turno activo en esa fecha y hora. En la base de datos se recomienda añadir una restricción de unicidad (Unique Constraint) compuesta por las columnas medico_id, fecha y horario para que el mismo motor impida el duplicado.

Criterio de aceptación: Al intentar mandar una segunda reserva idéntica, el backend debe capturar la restricción, rechazar el POST con un código de error (ej. 409 Conflict) y el frontend debe mostrar un mensaje claro: 'El horario seleccionado ya no se encuentra disponible'.

Riesgos de regresión: Se podría afectar el módulo de cancelación de turnos (que al liberar el turno falle la restricción) o el módulo de edición/reprogramación de turnos.

Paso 5 - Análisis de Causa Raíz (5 Whys)

1. ¿Por qué el sistema guardó dos turnos iguales? Porque el backend procesó el segundo POST sin rechazarlo.
2. ¿Por qué el backend procesó el segundo POST? Porque no ejecutó ninguna validación de disponibilidad antes de hacer el INSERT.
3. ¿Por qué no ejecutó la validación? Porque la función encargada de procesar las reservas no incluye el método de verificación previa de disponibilidad.
4. ¿Por qué no se incluyó ese método? Porque en la especificación técnica original no se detalló explícitamente cómo manejar la concurrencia o bloqueos de horarios ocupados.
5. **Causa Raíz:** Falta de definición en los requisitos técnicos de la arquitectura sobre cómo manejar la concurrencia y ausencia de restricciones únicas a nivel de base de datos en el diseño inicial.

Paso 6 - Plan de Re-testing

Ejecutar el caso de prueba original: Iniciar sesión, elegir Dra. Molina el 12/11/2025 a las 10:00 y confirmar. El sistema debe dar éxito. Inmediatamente, intentar repetir la operación exacta con el mismo usuario o uno diferente. Resultado esperado correcto: El sistema debe rebotar la segunda reserva, mostrar alerta de horario ocupado y no duplicar nada en la base de datos.

Paso 7 - Plan de Regresión

1. Probar cancelación de turnos: Reservar un turno, cancelarlo y verificar que el horario vuelva a estar disponible para reservas nuevas.
2. Probar visualización de agenda: Verificar que en el calendario del médico, los turnos ya tomados aparezcan bloqueados o grises para los pacientes.
3. Probar reprogramación: Intentar cambiar un turno existente a un horario libre y a uno ya ocupado, validando que el sistema responda bien en ambos casos.

Paso 8 - Ticket

ID:	TK-001
Título:	Permite reserva duplicada de turnos para el mismo médico y horario
Resultado esperado:	El sistema debe impedir la reserva de un horario que ya fue asignado.
Resultado obtenido:	El sistema confirma dos reservas idénticas y genera duplicados en la base de datos.
Estado:	Fail
Evidencia:	Log: POST /reservas - usuario 182 - OK (duplicado). Dos registros idénticos en la base.
Tipo de defecto:	Error lógico / Validación
Severidad:	Alta
Prioridad:	Alta
Propuesta de corrección:	Agregar verificación de disponibilidad en backend y Unique Constraint compuesta en la Base de Datos.
Causa raíz:	Ausencia de control de concurrencia y falta de restricciones de unicidad en el modelo de base de datos.
Plan de re-testing:	Ejecutar doble reserva idéntica consecutiva. La segunda debe ser rechazada.
Plan de regresión:	Verificar cancelación de turnos, visualización de grilla horaria y reprogramación.
Fecha de registro:	19/06/2026

CASO 2 - "Formulario de registro acepta email inválido"

Paso 1 - Interpretación del resultado

La evidencia muestra que la interfaz de registro de usuarios no tiene controles de formato sobre los datos de entrada. El sistema permitió crear una cuenta usando un correo totalmente erróneo (laura.perez@@gmail..com) sin alertar al usuario. Esto deriva en que la aplicación rompe más adelante en un flujo secundario crítico: cuando el sistema intenta disparar el mail de verificación, el servidor de correos (SMTP) falla silenciosamente porque la dirección no es válida.

Estamos ante una **falla** cuando el usuario intenta loguearse y queda bloqueado sin poder verificar su cuenta. El **defecto** está en el script del formulario del frontend y en el controlador del backend que no aplican una expresión regular (Regex) para validar correos. El **error** fue del programador al dejar desactivada o saltada la validación, tal como delata el log.

Paso 2 - Clasificación del defecto

Es un **Error de validación** (e Interfaz UI/UX). Justificación: El log técnico expone de forma directa la frase 'Email validation skipped for field email'. Esto significa que la rutina encargada de filtrar que el texto contenga un formato usuario@dominio.com fue omitida, permitiendo el ingreso de basura a la base de datos.

Paso 3 - Severidad y prioridad

- **Severidad: Media.** El impacto técnico no tira el servidor ni rompe la base de datos, los datos se guardan (aunque sean inválidos). Sin embargo, degrada la experiencia de usuario ya que bloquea el flujo posterior de inicio de sesión de ese registro en particular.
- **Prioridad: Media.** Aunque es molesto y traba al usuario, no impide el funcionamiento general del sitio para los usuarios que se registran bien. Se debe solucionar en el sprint actual para evitar acumular registros 'muertos' en la base de datos.

Paso 4 - Propuesta de acción correctiva

Se debe implementar la validación tanto en el Frontend como en el Backend. En el Frontend, aplicar un atributo type='email' al input HTML o usar una expresión regular en Javascript que valide el formato antes de habilitar el botón de envío. En el Backend, reestablecer y activar el validador del campo email antes de realizar la inserción, para asegurar que si un atacante saltea el front, el backend rechace el dato sucio.

Criterio de aceptación: Si el usuario ingresa un mail sin arroba, con doble arroba o puntos consecutivos, el formulario debe ponerse en rojo y mostrar el texto: 'Por favor, introduce una dirección de correo válida'. El botón de registrar debe quedar deshabilitado.

Riesgos de regresión: Ninguno crítico, pero hay que revisar que no afecte al formulario de login o al de recuperar contraseña.

Paso 5 - Análisis de Causa Raíz (5 Whys)

1. ¿Por qué se guardó un email inválido? Porque el backend no lo rechazó.
2. ¿Por qué el backend no lo rechazó? Porque la validación del email fue saltada por el sistema.
3. ¿Por qué se saltó la validación? Porque algún desarrollador puso una bandera o comentó la validación en el código de producción.
4. ¿Por qué se comentó o desactivó la validación? Porque durante la etapa de pruebas locales de desarrollo, les molestaba tener que escribir mails reales y desactivaron la función para avanzar rápido, olvidando reactivarla al subir el código.
5. **Causa Raíz:** Falta de controles o checklists de revisión de código (Code Review) antes de mezclar las ramas de desarrollo hacia la rama principal de producción.

Paso 6 - Plan de Re-testing

Ir al formulario de registro. Intentar crear una cuenta con el mail `laura.perez@@gmail..com`.
Resultado esperado correcto: El sistema no debe procesar el registro, debe pintar el campo con un aviso de error y bloquear el envío.

Paso 7 - Plan de Regresión

1. Probar login de usuarios existentes: Confirmar que los usuarios reales que tienen mails válidos inicien sesión normalmente.
2. Probar formulario de 'Olvidé mi contraseña': Verificar que valide correctamente el formato de correo antes de intentar enviar el token de recuperación.
3. Probar actualización de perfil: Ingresar a la configuración de un usuario y validar que el campo de cambiar correo use las mismas reglas estrictas de validación.

Paso 8 - Ticket

ID:	TK-002
Título:	Formulario de registro acepta correos con formatos inválidos sin validar
Resultado esperado:	El formulario y el backend deben rechazar correos con sintaxis incorrecta.
Resultado obtenido:	El sistema registra la cuenta y luego falla el flujo de verificación SMTP.
Estado:	Fail
Evidencia:	Log: [WARN] Email validation skipped for field 'email' y error de dirección inválida en SMTP.
Tipo de defecto:	Error de validación
Severidad:	Media
Prioridad:	Media
Propuesta de corrección:	Implementar y activar validación por Regex tanto en frontend (UI) como en el controlador de backend.
Causa raíz:	Omitir o comentar la línea de validación durante desarrollo y falta de revisión de código previa al despliegue.
Plan de re-testing:	Cargar un correo mal estructurado en el registro; el sistema debe lanzar alerta y bloquear el POST.

Plan de regresión:	Testear inicio de sesión normal, recuperación de contraseña y cambio de email en el perfil.
Fecha de registro:	19/06/2026

CASO 3 - "Certificados PDF con acentos mal impresos"

Paso 1 - Interpretación del resultado

La evidencia nos muestra un claro problema de codificación de caracteres en la librería o componente que genera los archivos PDF. En la pantalla de la web no hay errores y el flujo termina bien (el archivo se descarga), pero al abrir el PDF se observa que los caracteres con tildes o eñes se transformaron en símbolos extraños (ej. 'María González' sale como 'Maria GonzÃ¡lez').

Esto representa una **falla** en la salida final entregada al usuario (documento ilegible/impresentable). El **defecto** está en la configuración de la fuente o en el encoding del flujo de datos del backend que genera el PDF, que no procesa correctamente la codificación UTF-8. El **error** es del programador por no configurar de manera correcta el soporte de caracteres en español dentro de la herramienta de renderizado de PDFs.

Paso 2 - Clasificación del defecto

Es un **Error de interfaz / representación (UI/UX)** o error de configuración técnica.

Justificación: Funcionalmente el archivo se genera, pero los datos se rompen en su capa de presentación visual. El log da la pauta clave: '[WARN] Fallback de fuente por incompatibilidad UTF-8', lo que significa que el generador del PDF no tiene una fuente tipográfica capaz de mapear caracteres especiales en UTF-8 y pone símbolos genéricos reemplazando el texto correcto.

Paso 3 - Severidad y prioridad

- **Severidad: Media.** Desde el punto de vista técnico estricto, el sistema no está roto, la base de datos está intacta y el flujo no se interrumpe. Sin embargo, el producto final no sirve para su propósito principal.
- **Prioridad: Alta.** Contexto de negocio: Hay un acto de colación la semana que viene donde se tienen que imprimir más de 500 certificados oficiales frente a autoridades institucionales. Entregar un título oficial con el nombre roto daña severamente la imagen de la universidad y obligaría a gastar presupuesto extra reimprimiendo todo a las apuradas. La opinión del responsable de sistema ('es solo un tema de acentos') ignora por completo el impacto y riesgo del negocio. Debe arreglarse ya.

Paso 4 - Propuesta de acción correctiva

Se debe modificar la configuración del módulo generador de PDF en el backend. Hay que

forzar explícitamente el uso de codificación UTF-8 en el flujo de renderizado de la página al PDF y, principalmente, asegurar que la fuente tipográfica embebida en el PDF (ej. Arial o Helvética) tenga cargado el set de caracteres latinos correspondientes para interpretar acentos y la letra ñe.

Criterio de aceptación: Al descargar el certificado de un alumno con caracteres especiales, el texto debe leerse perfectamente ('María González', 'Mendoza, República Argentina') sin alteraciones gráficas.

Riesgos de regresión: Podría alterarse la alineación del texto, el espaciado o los márgenes del PDF al cambiar el archivo de fuente tipográfica.

Paso 5 - Análisis de Causa Raíz (Ishikawa / Espina de Pescado)

Haciendo un análisis simplificado de las causas que confluyen en este defecto:

- **Personal:** El desarrollador probó la generación de PDFs usando nombres sin acentos (ej. 'Juan Perez' sin tilde) y dio por buena la funcionalidad sin hacer testing con datos extremos o locales.
- **Tecnología/Software:** La librería de PDF viene por defecto configurada en formato ISO-8859-1 o ASCII estándar estadounidense en vez de UTF-8.
- **Método:** No se incluyeron strings con caracteres especiales en el banco de datos de pruebas iniciales de QA para componentes visuales.
- **Causa Raíz:** Uso de configuraciones regionales por defecto (estilo USA) en las librerías de terceros instaladas y falta de pruebas unitarias específicas con caracteres del idioma español.

Paso 6 - Plan de Re-testing

Generar y descargar el PDF para el estudiante 'María González'. Abrir el archivo y verificar visualmente que el nombre y el texto fijo ('República Argentina') muestren las tildes correctas y no símbolos raros.

Paso 7 - Plan de Regresión

1. Probar certificados con nombres largos o complejos para comprobar que el cambio de fuente no rompa los márgenes ni genere desbordes de texto en el diseño del PDF.
2. Verificar el flujo de descarga en diferentes navegadores (Chrome, Firefox, Safari) para cerciorarse de que el archivo conserve sus propiedades tipográficas.
3. Revisar otros listados o reportes exportables en PDF que tenga el sistema para garantizar que no se hayan roto por el cambio en la configuración global del codificador.

Paso 8 - Ticket

ID:	TK-003
Título:	Caracteres especiales y acentos se rompen (mal encoding) al generar certificados PDF
Resultado esperado:	El PDF debe mostrar correctamente los acentos y ñes ('María González', 'República').

Resultado obtenido:	Los acentos se reemplazan por símbolos extraños (ej. 'RepÃ blica').
Estado:	Fail
Evidencia:	Log: [WARN] Fallback de fuente por incompatibilidad UTF-8. Certificados descargados legibles con errores de caracteres.
Tipo de defecto:	Error de Interfaz / Configuración de Encoding
Severidad:	Media
Prioridad:	Alta
Propuesta de corrección:	Configurar codificación UTF-8 embebida e instalar / referenciar fuentes con mapa de caracteres latinos en la librería generadora de PDF.
Causa raíz:	Librería de PDF con tipografía por defecto incompatible con UTF-8 y omisión de pruebas con caracteres en español.
Plan de re-testing:	Emitir certificado para usuarios con tildes y eñes, confirmando la correcta visualización del documento.
Plan de regresión:	Controlar alineación general del PDF, probar descargas en varios navegadores y chequear otros reportes PDF del sistema.
Fecha de registro:	19/06/2026

CASO 4 - "Carrito de compras recalcula mal el total después de aplicar un descuento"

Paso 1 - Interpretation del resultado

La evidencia nos muestra un error matemático en las operaciones del carrito de compras. Cuando el usuario aplica un código de descuento del 10% sobre un subtotal de \$30.000, el total resultante en pantalla debería ser de \$27.000. Sin embargo, el sistema calcula y muestra \$24.000 (restando un 20% en lugar del 10% estipulado). Lo alarmante es que en la base de datos se guarda el total incorrecto (\$24.000) pero la etiqueta promocional dice 0.10 (10%). Estamos ante una **falla** funcional crítica (pérdida económica directa en cada transacción). El **defecto** reside en la fórmula o en la función de cálculo matemático dentro del backend. El **error** se originó, según avisa el desarrollador, cuando tocaron una función de cálculo de precios para

otra sección la semana pasada e introdujeron una regresión en este módulo.

Paso 2 - Clasificación del defecto

Es un **Error lógico** de regresión. Justificación: La interfaz muestra la información que recibe de forma limpia, pero la matemática interna encargada de procesar el descuento está errada. El log de debug confirma que el porcentaje cargado es correcto (`discount_percent = 0.1`) pero el total asignado final salta a 24000 de manera errónea, evidenciando un cálculo mal planteado en los métodos del backend.

Paso 3 - Severidad y prioridad

- **Severidad: Crítica.** Bloquea o afecta directamente el núcleo del negocio (módulo de pagos). Al cobrar de menos, genera pérdidas económicas directas y automatizadas al comercio, además de descuadres contables graves y problemas de auditoría difíciles de solucionar de forma retroactiva.
- **Prioridad: Alta / Inmediata.** El costo de postergación es prohibitivo para el negocio. Cada minuto que pase con este bug activo significa dinero perdido y registros contables arruinados. Debe corregirse de inmediato deteniendo cualquier otra tarea.

Paso 4 - Propuesta de acción correctiva

Se debe auditar y corregir la lógica matemática de la función de cálculo de totales en el backend. Revisar qué cambios se hicieron la semana pasada y deshacer la alteración que afectó la fórmula del carrito. La fórmula debe asegurar un cálculo simple: $\text{total} = \text{subtotal} - (\text{subtotal} * \text{discount_percent})$.

Criterio de aceptación: Al sumar productos por \$30.000 y meter el cupón de 10%, el total debe ser exactamente \$27.000 tanto en la pantalla como en el registro final de la base de datos.

Riesgos de regresión: Alto riesgo de romper el otro módulo de precios que intentaron arreglar la semana pasada. Se debe verificar con especial cuidado.

Paso 5 - Análisis de Causa Raíz (5 Whys)

1. ¿Por qué el cliente pagó de menos? Porque el total cobrado y guardado fue de \$24.000 en vez de \$27.000.
2. ¿Por qué el total fue de \$24.000? Porque la función matemática del carrito restó el doble del porcentaje correspondiente.
3. ¿Por qué la función restó el doble? Porque la semana pasada se modificó el archivo de cálculos compartidos para solucionar otro módulo y se alteró la lógica global de descuentos.
4. ¿Por qué se alteró la lógica global si era para otro módulo? Porque la función no estaba correctamente modularizada o aislada, provocando que un cambio en un lado impactara en otro de manera colateral.
5. **Causa Raíz:** Acoplamiento excesivo en el código fuente (funciones compartidas sin aislamiento adecuado) y falta de ejecución de pruebas de regresión automáticas antes de subir la actualización de la semana pasada.

Paso 6 - Plan de Re-testing

Armar un carrito con los mismos montos (\$10.000, \$8.000, \$12.000), subtotal \$30.000. Aplicar cupón 'DESCUENTO10'. Resultado esperado correcto: El total debe mostrar \$27.000 en la UI y el registro en la base de datos debe almacenar total: 27000 y descuento_aplicado: 0.10.

Paso 7 - Plan de Regresión

1. Probar el otro módulo de precios (el que arreglaron la semana pasada) para verificar que la nueva corrección no vuelva a romper su funcionalidad.
2. Probar el carrito de compras sin cupones de descuento, asegurando que el total sea la suma exacta de los productos sin alteraciones misteriosas.
3. Probar la inserción de múltiples cupones válidos, inválidos o vencidos para validar que el sistema aplique el porcentaje cero o el descuento correspondiente según el estado del cupón.

Paso 8 - Ticket

ID:	TK-004
Título:	El carrito calcula erróneamente un 20% de descuento en lugar del 10% fijado por el cupón
Resultado esperado:	Subtotal \$30.000 con 10% de descuento debe dar un total de \$27.000.
Resultado obtenido:	El sistema calcula un total de \$24.000 (aplica un 20%) y lo graba erróneamente en la base de datos.
Estado:	Fail
Evidencia:	Logs de Debug revelan subtotal=30000, discount=0.1, pero total=24000 de forma inconsistente.
Tipo de defecto:	Error lógico / Regresión
Severidad:	Crítica
Prioridad:	Alta
Propuesta de corrección:	Corregir la fórmula matemática en la función compartida del backend y aislar la lógica para evitar efectos colaterales.
Causa raíz:	Código acoplado modificado recientemente por otro requerimiento sin controles de regresión.

Plan de re-testing:	Efectuar la misma compra de \$30.000 con cupón del 10% y auditar que el resultado sea \$27.000 exactos en pantalla y base de datos.
Plan de regresión:	Validar el módulo modificado la semana pasada, chequear carritos sin descuentos y probar cupones inválidos/expirados.
Fecha de registro:	19/06/2026